Beavers Choice Paper Company

Agentic AI Workflow Solution

Problem Statement

The Beaver's Choice Paper Company is struggling with managing their paper supplies, responding promptly to customer inquiries, and generating competitive quotes. They're overwhelmed and losing potential sales due to inefficiencies. Your challenge is to design and implement a multi-agent solution, restricted to at most five agents, capable of handling inquiries, checking inventory status, providing accurate quotations, and completing transactions seamlessly. Your solution must ensure responsiveness, accuracy, and reliability in managing requests and maintaining optimal stock levels.

Proposal

Develop a multi-agent system that streamlines operations, enables quick quote generation, accurate inventory tracking, and ultimately drive increased sales.

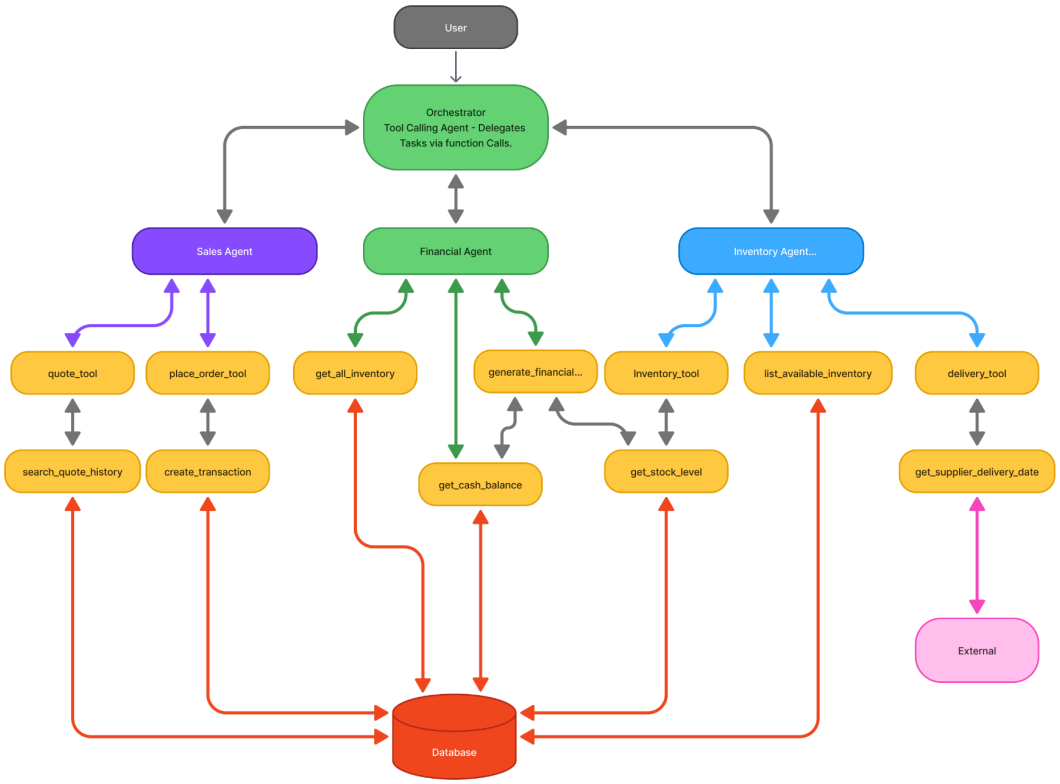
Multiagent design

Munder Difflin Multi-Agent System Design

Executive Summary The Munder Difflin Multi-Agent System is a sophisticated automation solution designed to modernize the workflows of a paper manufacturing enterprise.

By leveraging a hierarchical multi-agent architecture built on the smolagents framework, the system autonomously handles inventory management, complex quote generation with bulk discounting, and order fulfillment.

The solution ensures data integrity through a centralized SQLite database and provides real-time financial tracking and reporting.



System Architecture

Orchestration Framework

- The system utilizes the smolagents library, specifically the ToolCallingAgent class. The orchestrator delegates tasks to specialized agents via function calls, while each agent is restricted to its own defined set of tools.

Agent Roles and Responsibilities

- The system is comprised of three primary agents:
    - **Orchestrator (ToolCallingAgent):** Acts as the "brain" of the operation. It receives natural language requests from customers, decomposes them into logical steps, and delegates tasks to the appropriate specialized agent. It synthesizes the final response for the user.

    - **Inventory Agent (ToolCallingAgent):** Specialized in supply chain logistics. It has access to real-time stock levels and supplier lead times.
        - Tools: inventory_tool (wraps get_stock_level), delivery_tool (wraps get_supplier_delivery_date), get_all_inventory_tool.

    - **Sales Agent (ToolCallingAgent):** Specialized in customer relations and financial transactions. Handles pricing logic and database entry for sales.
        - Tools: quote_tool (wraps search_quote_history), place_order_tool (wraps create_transaction).

    - **Financial Agent (ToolCallingAgent):** Specialized in financial reporting and inventory valuation. Provides a real-time view of company assets and cash position.
        - Tools: get_all_inventory, get_cash_balance, generate_financial_report (internally calls get_cash_balance and get_stock_level).

Data Infrastructure

- **Database:**
    - A local SQLite database (munder_difflin.db) serves as the single source of truth for inventory, historical quotes, and financial transactions.
- **Utility Layer:**
    - A set of Python helper functions (create_transaction, search_quote_history, get_stock_level, get_supplier_delivery_date, get_cash_balance, get_all_inventory, generate_financial_report) abstracts complex SQL operations into simple callable functions for the agents. Note: only ~40% of catalog items are stocked by design — "product not found" responses are expected behavior, not errors.

Design Decisions

The chosen implementation was a **Hierarchical Delegation with a Manager-Worker pattern** was implemented. The Alternative being a flat structure where all agents can talk to each other)

The **Orchestrator** manages the state and sequence, preventing "agent loops" and ensuring that the system follows a logical business process: Stock Check → Quote → Order Placement.

Tool-Centric Agents do not have direct access to the database. Instead, they interact with the data through dedicate tools. This ensures that the LLM cannot execute arbitrary SQL, maintaining data security and structural integrity.

Intelligence-Driven Quoting The quote_tool was designed to be "context-aware." It doesn't just calculate a price; it performs a vector-like search (using keywords) on search_quote_history. This allows the agent to justify its pricing based on past successful deals, mirroring a real-world sales professional's behavior.

Scalable Discount Logic An example tiered bulk discount strategy was implemented within the *quote_tool*:

- 0-99 units: Base price
- 100-499 units: 5% discount
- 500-999 units: 10% discount
- 1000+ units: 15% discount

NOTE: *This needs to be adjusted to the business requirements for bulk discounts.*

Meeting the Project Requirements

1. Automation of Core Operations
    - **Inventory Checks**: The inventory_agent automatically flags when stock falls below min_stock_level and provides estimated restock dates.
    - **Quote Generation**: Quotes are generated dynamically based on real-time unit prices and the bulk discount logic mentioned above.
    - **Order Fulfillment**: Transactions are recorded automatically in the transactions table upon order placement, updating the company's cash balance and inventory levels.
2. Constraints and Performance
    - **Agent Limit**: The system uses 3 agents, well within the 5-agent maximum requirement.
    - **Communication**: All interactions are entirely text-based, allowing for easy integration with chat interfaces or email systems.

- **Financial Integrity**: The generate_financial_report function provides a comprehensive view of Cash, Inventory Value, and Total Assets, ensuring the system remains financially accountable.

3. **Data Utilization** The system successfully integrates historical data from quotes.csv and quote_requests.csv into the runtime environment, allowing the AI to learn from and reference "Munder Difflin" history.

Evaluation Results Analysis

The system's performance was evaluated against a diverse set of procurement requests. The following table summarizes representative interactions and their outcomes:

Request ID	Request Detail	System Response Summary	Status
1	Ceremony supplies (Colored paper, A4 glossy, Heavy cardstock).	Successfully placed all orders with delivery by April 15.	Success
3	Conference requirements (A3 paper).	Identified A3 stockout and suggested A4/Cover stock alternatives.	Failure (Partial)
18	Standard printing paper and high-quality white cardstock.	Insufficient stock for printer paper; failed to check cardstock availability.	Failure
19	Exhibition supplies (A4 glossy, A3 matte, Assorted cardstock).	Identified partial stock for A4 and cardstock; zero stock for A3. Requested user guidance.	Success (Inquiry)

System Strengths

The system demonstrates an improved ability to suggest internal alternatives when a specific item (like A3 paper) is unavailable, showing a better grasp of product relationships. Additionally, in the April 15th exhibition request (ID 19), the system correctly identified partial stock levels

rather than providing a binary "available/unavailable" response, allowing for more nuanced decision-making.

Observed Failures

A persistent logic error remains where the system occasionally halts inventory checks for secondary items if the primary item in a request is out of stock (Request 18). Furthermore, while the system identifies when a delivery date will miss a deadline (Request 3), it does not yet proactively suggest expedited shipping or alternate vendors to bridge the gap.

Conclusion

The implemented solution provides a robust, scalable, and intelligent multi-agent system capable of handling the complexities of paper sales and logistics. By combining modern LLM orchestration with traditional relational database management, the system offers a reliable bridge between natural language business requests and structured data execution.

Iterating on this foundation and providing a full inventory database, would see all requirements met.

Improvement Suggestions

To meet the full operational requirements and address the limitations noted during testing, the following improvements are proposed:

- **Implementation of a full database:** Currently the system has access to a subset of inventory as a prototype. The final implementation will need to be tested with a full database to ensure that it operates as expected.
- **Implementation of a Restock-Ordering Agent:** Currently, the system identifies when inventory is insufficient but requires manual intervention to initiate a reorder. A dedicated agent should be integrated to automatically generate purchase orders with pre-approved suppliers when stock falls below a defined threshold, ensuring continuity for high-demand items like A4 glossy paper.
- **NLP Parsing for Multi-Item and Ambiguous Requests:** To reduce failures related to "missing information," the natural language processing (NLP) layer should be upgraded to handle complex, multi-part requests more robustly. This includes the ability to suggest alternatives for out-of-stock items or prompt the user for clarification when specific attributes (like paper weight or finish) are missing, rather than simply failing the request.
- **Discount Logic and Pricing Integration:** As previously noted, the discount logic requires adjustment to handle volume-based pricing. Ensuring the discount meets the business requirements, and or Integrating a live pricing API for more dynamic pricing will prevent the system from stalling when static pricing data is unavailable.
- **Implementation of Parallel Inventory Checking:** The logic should be refactored to ensure that the status of every item in a multi-part request is checked independently. The failure or stockout of one item should not preclude the system from providing stock and delivery data for the remaining items, as seen in the failure of Request 18.
- **Automated Vendor Expansion/Expedited Logistics:** When a primary supplier cannot meet a deadline (as occurred in Request 3), the system should include a "logistics recovery" step. This would involve automatically checking for alternate suppliers or calculating the cost-benefit of expedited shipping options to ensure event deadlines are

met.

- **Dynamic Quantity Adjustment Prompting:** Building on the partial success of Request 19, the system should be enhanced to automatically ask if the user wants to "ship what is available now" while placing the remainder on backorder, rather than just identifying the shortage.